Good — you fixed the *first problem* (duplicate classes), but now you've hit the **next level of Java** 😄 — packages must match folders.

❌ Your current error

> **"The declared package `inheritance` does not match the expected package ""„**

🔥 What this actually means

You wrote:

package inheritance;

But your file is NOT in the correct folder structure for that package.

🧠 Core Java Rule (VERY IMPORTANT)

> **Package name must match the folder structure exactly**

📂 What I see in your screenshot

Your file is here:

```
Lecture-16-Encapsulation-and-Inheritance/
  code/
    inheritance/
      Inheritance.java
```

So the **correct package should be:**

package code.inheritance;

❌ Why your current code fails

You wrote:

package inheritance;

But Java expects:

- either file in `/inheritance/Inheritance.java`
- OR package to reflect full path

Right now mismatch = ❌ error

✅ Fix (choose ONE)

✅ Option 1 (BEST): Fix package name

Change this:

package inheritance;

👉 to:

package code.inheritance;

✅ Option 2: Move folder

Move `inheritance` folder to root:

```
Coder-Army-Java/
  inheritance/
    Inheritance.java
```

Then your original works:

package inheritance;

⚠️ ALSO IMPORTANT (hidden issue)

You have:

Student.class
EngineeringStudent.class

👉 These must ALSO be in the same package OR imported properly

---

🧠 Mental Model (very important)

Think like this:

Folder Path	Package Name
`code/inheritance`	`code.inheritance`
`com/app/service`	`com.app.service`

👉 `/` becomes `.`

🚀 Why Java is strict here

Because Java uses packages to:

- avoid naming conflicts (your earlier problem 👀)
- organize large systems
- enable imports cleanly

⚡ Quick sanity check

Ask yourself:

> "If I convert folder path to dots, does it match my package?"

If not → ❌ error

🧩 Final insight (logic coach mode)

You solved:

- ❌ Duplicate class names → with packages

Now you're learning:

- ❌ Package ≠ random label
- ✅ Package = folder structure identity

If you want, next I can:
👉 clean your whole project structure (like a real-world Java repo)
👉 show how imports work between `Student`, `EngineeringStudent`, etc.

The "declared package does not match expected package" Error

What's happening

Your file has this at the top:

package inheritance;

This tells Java: **"This class belongs to a package called `inheritance`"**

How Java Packages Work

When you declare a package, Java **strictly requires** the folder structure to match. It's a rule:

package inheritance; → file MUST be inside a folder named "inheritance"
package com.myapp; → file MUST be inside com/myapp/
package animals.dog; → file MUST be inside animals/dog/

Think of it like a **home address** — if your ID says you live at "123 Main St" but you're actually at "456 Oak Ave", there's a mismatch. 🏠

Your Situation

Looking at your file path in VS Code:

Lecture-16-.../code/inheritance/Inheritance.java

Your file **IS** inside the `inheritance` folder ✅

So why the error? Again — **the Language Server** is the culprit. It's treating your whole `Coder-Army-Java` as one project, and getting confused about where the "root" of the project is.

The Fix — Click "Fix" button

VS Code is already showing you the solution! In the popup you can see:

Fix (⌘I) or Fix

Just click **Fix** — it will either:

- Remove the `package inheritance;` line, OR
- Set the correct package path

Should you even use `package inheritance`?

For a **learning/practice project** like Coder Army lectures, you likely **don't need package declarations at all**.

Just **delete line 4**:

// DELETE THIS ↓
package inheritance;

Your code will run perfectly fine without it, since you're just practising OOP concepts — not building a real multi-module application.

Quick Summary

	Explanation
Error cause	`package` name must match folder structure
Why it's confusing	Language Server gets the root path wrong
Quick fix	Click the Fix button VS Code shows
Best fix for learning	Just remove the `package` declaration

Good — now we're very close. This is the **final subtle issue**, and it's not your logic — it's **VS Code Java configuration**.

❌ Current error

> **"The declared package `code.inheritance` does not match the expected package """"**

This means:

👉 Java is still treating your file as being in the **root (default package)**
👉 NOT inside `code/inheritance` as you think

🔥 Root Cause (important)

VS Code **does NOT consider your `code/` folder as a source folder**

So internally it thinks:

Inheritance.java → (no package)

But you wrote:

package code.inheritance;

👉 mismatch = ❌ error

🧠 This is NOT a Java problem — it's a project structure problem

Java expects something like:

```
project-root/
  src/
    code/
      inheritance/
        Inheritance.java
```

✅ Fix (2 clean ways)

✅ Option 1 (BEST — industry standard)

Step 1: Create `src` folder

```
Coder-Army-Java/
  src/
    code/
      inheritance/
        Inheritance.java
```

Step 2: Move ALL your `.java` files inside `src`

Step 3: Tell VS Code

- Press `Cmd + Shift + P`
- Search: **Java: Configure Classpath**
- Add `src` as source folder

👉 Now your package works:

package code.inheritance;

✔ error gone

✅ Option 2 (Quick hack — not recommended long term)

Remove package:

// ❌ remove this
package code.inheritance;

✔ Works
❌ But brings back your original duplicate class problem

⚠️ Also I noticed something IMPORTANT

You have:

EngineeringStudent.class
Student.class

👉 These are `.class` files (compiled output)

❌ Problem:

They are sitting inside your source folder

✅ Fix:

Delete them manually or:

rm *.class

👉 Java will regenerate them

🧠 Mental model (this is the key learning)

Right now your brain says:

> "Folder = structure"

But Java says:

> "Only folders inside SOURCE PATH matter"

🚀 Final clean structure (what you should aim for)

```
Coder-Army-Java/
  src/
    lecture16/
      inheritance/
        Inheritance.java
        Student.java
        EngineeringStudent.java
```

package lecture16.inheritance;

⚡ Quick diagnosis question (for you)

Before writing `package X.Y`, always ask:

> "Is my file inside a **source folder** that matches X/Y?"

If you want next step

I can:

- Fix your entire repo structure cleanly
- Show how imports work (`import lecture16.inheritance.Student`)
- Help you turn this into a **real Java project (Maven/Gradle)**

Just say 👍

What Happened

When you clicked **Fix**, VS Code automatically changed your package declaration from:

package inheritance;

to:

package code.inheritance;

This is because the Language Server now thinks the root of your project is `Lecture-16-.../`, so it sees the path as `code → inheritance`, hence `code.inheritance`.

But it's **still wrong** because it can't figure out the correct root. It will keep guessing incorrectly.

The Permanent Fix

Just delete the package line entirely. Since you're learning/practising, you don't need it at all.

Change your file to:

```
/*
  Inheritance
*/
```

```
public class Inheritance {

    public static void main(String[] args) {
        EngineeringStudent es = new EngineeringStudent();
        es.markAttendance();
        es.attendLab();

        Student s1 = new Student();
        s1.markAttendance();
        // s1.attendLab(); // (wrong)
    }
}
```

Just **remove line 4 completely** (`package code.inheritance;`) — that's it. ✅

Why you don't need `package` here

Packages are only needed when:

- You're building a **large real-world project**
- You have **hundreds of classes** to organize
- You're publishing a **library**

For **Coder Army lecture practice files** → packages are completely unnecessary. Your instructor likely added it just to demonstrate the concept, not because it's required to run the code.

Now I can see the exact mistake 👇 — this one is simple but **very important Java rule**.

❌ Your current line (WRONG)

package Lecture-16-Encapsulation-and-Inheritance.code.inheritance;

🔥 Why this is WRONG

Java package names **CANNOT contain hyphens** `-`

👉 `Lecture-16-Encapsulation-and-Inheritance` ❌ invalid

Because Java treats `-` as a **minus operator**, not part of a name.

🧠 Java package naming rules

Allowed:

✔ lowercase letters
✔ numbers
✔ dots (.)

NOT allowed:

❌ hyphens (-)
❌ spaces
❌ special characters
❌ uppercase (not recommended)

✅ Fix (correct it like this)

Change:

package Lecture-16-Encapsulation-and-Inheritance.code.inheritance;

👉 to something clean:

package lecture16.inheritance;

📂 Then your folder MUST match

Right now you have:

Lecture-16-Encapsulation-and-Inheritance/code/inheritance

👉 This is still ❌ wrong for Java

✅ Correct folder structure

Rename folders like this:

```
lecture16/
  inheritance/
    Inheritance.java
```

🚀 Final working combo

Folder:

lecture16/inheritance/Inheritance.java

Code:

package lecture16.inheritance;

✔ Now Java is happy
✔ VS Code error gone

⚠️ ALSO — still pending from before

You STILL have:

Demo.java repeated everywhere

👉 That error will remain until you:

- use packages everywhere **OR**
- rename classes

🧠 Logic coach moment

You tried:

> "Let me match folder name exactly"

Good instinct ❗
But Java expects:

> "Convert folder → valid identifier"

💡 Pro tip (used in real companies)

Package naming style:

com.company.project.module

For you:

com.hari.lecture16.inheritance

⚡ Quick checklist

Before writing `package`:

- ❓ Does it have `-` → remove it
- ❓ Does it match folder → yes
- ❓ Is it lowercase → yes

If you want, next step I'll:
👉 clean your entire repo into a **professional Java project structure** (this will remove ALL your current errors permanently)

It keeps getting worse! 😅 VS Code's auto-fix is making it worse each time — now it's generated:

package Lecture-16-Encapsulation-and-Inheritance.code.inheritance;

which is completely invalid (hyphens `-` are not allowed in package names).

Stop clicking Fix! Here's what to do:

Manually delete the entire package line yourself.

1. Click at the beginning of line 4 (the `package` line)
2. Select the **entire line**
3. Press **Delete/Backspace**

Your file should look like this:

```
/*
  Inheritance
*/

public class Inheritance {

   public static void main(String[] args) {
      EngineeringStudent es = new EngineeringStudent();
      es.markAttendance();
      es.attendLab();

      Student s1 = new Student();
      s1.markAttendance();
      // s1.attendLab(); // (wrong)
   }
}
```

No `package` line at all. ✅

Golden Rule going forward

> ❌ **Never click "Fix"** on package-related errors in VS Code when you're doing lecture practice files.

VS Code's fix just keeps guessing the wrong package path and makes things worse every time. Always **manually delete** the package line instead.